🪝 React Native Master Guide — Level 3 (Complete Verbatim Edition)

Mobile Screen Lifecycle, Memory Leaks, Performance Optimization (useMemo / useCallback), Imperative useRef & Custom Hooks

EXPO V54 • NATIVEWIND / TAILWIND CSS • ES6+

🪝 Level 3.1: Mobile Screen Lifecycle & useEffect Cleanup

1. Mental Model: Why Cleanup Functions are Critical in Mobile

Web browsers lo page refresh chesthe memory clear aypothundi. Kani mobile apps lo **Navigation Stack** untundi — user oka screen nundi vere screen ki vellina, patha screen background memory lo alage undocchu.

If you start a `setInterval`, GPS location tracker, or AppState listener in `useEffect` **without returning a cleanup function**:

- Timer background lo alage run avthu phone **battery ni drain chesthundi**.
- Component screen nundi poyaka kooda state update cheyadaniki try chesi **Memory Leak** create chesthundi!

```
Component Mounts  ──▶  Runs useEffect()  ──▶  Starts Timer / Listener ⏱️
                                                        |
User Navigates Away ──▶ Runs Cleanup return () => {} ──▶ Cancels Timer (Safe! ✅)
```

2. The 3 Lifecycle Phases of useEffect

Dependency Array	Phase	When it runs
`useEffect(() => {}, [])`	**Mount**	Screen first time open ayinappudu okkasari mathrame run avthundi (API calls, start timers).
`useEffect(() => {}, [count])`	**Update**	`count` value maarina prathi sari trigger avthundi.
`return () => { clearInterval(id); }`	**Unmount / Cleanup**	Screen close ayinappudu or component unmount ayinappudu timers/listeners ni destroy chesthundi.

💻 Code Example: Live OTP Resend Timer & Session Tracker

```
import React, { useState, useEffect } from 'react';
import { View, Text, Pressable } from 'react-native';
import { SafeAreaProvider, SafeAreaView } from 'react-native-safe-area-context';
import { StatusBar } from 'expo-status-bar';

const App = () => {
  const [secondsLeft, setSecondsLeft] = useState(30);
  const [isActive, setIsActive] = useState(true);
  const [resendCount, setResendCount] = useState(0);

  // 1. Lifecycle Hook with Mandatory Cleanup Function
  useEffect(() => {
    let timerInterval = null;

    if (isActive && secondsLeft > 0) {
      // Start 1-second interval
      timerInterval = setInterval(() => {
        setSecondsLeft((prev) => prev - 1);
      }, 1000);
    } else if (secondsLeft === 0) {
      setIsActive(false);
    }

    // 🚨 MANDATORY CLEANUP: Clears interval when component unmounts or state changes
    return () => {
      if (timerInterval) {
        clearInterval(timerInterval);
      }
    };
  }, [isActive, secondsLeft]); // Re-evaluates when isActive or secondsLeft changes

  // 2. Restart Timer Handler
  const handleResendOtp = () => {
    setSecondsLeft(30);
    setIsActive(true);
    setResendCount((prev) => prev + 1);
  };

  return (
    <SafeAreaProvider>
      <StatusBar style="light" backgroundColor="#090d16" />
      <SafeAreaView className="flex-1 bg-slate-950 px-6 justify-center">

        {/* Card Container */}
        <View className="bg-slate-900 border border-slate-800 p-6 rounded-3xl items-center shadow-2xl">

          <View className="w-16 h-16 rounded-2xl bg-sky-500/20 border border-sky-500/30 items-center justify-center mb-4">
            <Text className="text-3xl">🔐</Text>
          </View>

          <Text className="text-2xl font-black text-white mb-1">
            Verification Code
          </Text>
          <Text className="text-slate-400 text-xs text-center mb-6">
            We sent a 6-digit OTP to your registered phone number.
          </Text>

          {/* Dynamic Countdown Display */}
          <View className="bg-slate-950 border border-slate-800 px-6 py-4 rounded-2xl w-full items-center mb-6">
            <Text className="text-slate-400 text-xs font-semibold uppercase mb-1">
              Code Expires In
            </Text>
            <Text className={`text-3xl font-extrabold ${secondsLeft <= 5 ? 'text-red-400' : 'text-sky-400'}`}>
              00:{secondsLeft < 10 ? `0${secondsLeft}` : secondsLeft}
            </Text>
          </View>

          {/* Action Button */}
          <Pressable
            disabled={isActive}
            onPress={handleResendOtp}
```

```
            className={`w-full p-4 rounded-xl items-center ${
              isActive
                ? 'bg-slate-800 opacity-50'
                : 'bg-sky-500 active:bg-sky-600'
            }`}
          >
            <Text className="text-white font-bold text-sm">
              {isActive ? `Wait for timer...` : `Resend OTP Code 🔄`}
            </Text>
          </Pressable>

          {resendCount > 0 && (
            <Text className="text-slate-500 text-xs mt-3">
              Resent {resendCount} {resendCount === 1 ? 'time' : 'times'}
            </Text>
          )}

        </View>

      </SafeAreaView>
    </SafeAreaProvider>
  );
};

export default App;
```

🗣️ Telugu + English Explanation:

• **useEffect Cleanup**: Mobile lo `setInterval` or real-time event listeners pettinappudu, component unmount aythe `return () => clearInterval(timerInterval)` compulsory rayali. Idi rayakapothe user vere screen ki vellipoyina timer background lo alage run ayyi phone battery drain avthundi and memory leak vasthundi.

• **Dependency Array [isActive, secondsLeft]**: `secondsLeft` maarina prathi second ki effect re-check cheskuntundi. Once `0` reach avvagane timer ni stop chesthunnam.

🛠️ Quick Check & Cleanup Rules:

```
useEffect(() => {
  const subscription = Location.watchPositionAsync({}, (loc) => setCoords(loc));
  return () => {
    if (subscription) { subscription.then((sub) => sub.remove()); }
  };
}, []);
```

• `clearInterval(id)` ➔ for `setInterval()`

• `clearTimeout(id)` ➔ for `setTimeout()`

• `subscription.remove()` ➔ for Native Event Listeners, Location tracking, Sensors, and AppState.

🪝 Level 3.2: Performance Optimization (useMemo vs useCallback)

1. The Problem: Unnecessary Mobile Re-renders

React Native lo oka component state change ayinappudu (e.g. search bar lo user prathi letter type chesthunnappudu), **aa component function top nundi bottom varaku malli execute avthundi**.

Deeni valla 2 major performance issues vasthayi:

1. **Heavy Calculations**: 1,000 items unna array ni prathi keystroke ki malli filter & sort cheyadam valla UI freeze avthundi (Jank/Lag).
2. **Function Re-creation**: Prathi render lo functions ki kothha memory address (reference) create avthundi. Deeni valla child components & `FlatList` items anavasaram ga re-render avthayi.

2. The Solution: useMemo vs useCallback

Hook	What it Caches / Stores	Mental Rule
`useMemo`	**The RESULT / VALUE** of a calculation	*"Don't recalculate this filtered array unless `query` or `items` change!"*
`useCallback`	**The FUNCTION REFERENCE** itself	*"Don't create a new function in memory on every render unless `deps` change!"*

```
useMemo:
[Expensive Calculation] ──▶ Stores Result: [Item A, Item B] ──▶ Returns cached array ✅

useCallback:
const handleDelete = () => {} ──▶ Stores Function Reference in RAM ──▶ Prevents child re-renders ✅
```

💻 Code Example: High-Speed Product Search Engine

```
import React, { useState, useMemo, useCallback } from 'react';
import { View, Text, TextInput, FlatList, Pressable } from 'react-native';
import { SafeAreaProvider, SafeAreaView } from 'react-native-safe-area-context';
import { StatusBar } from 'expo-status-bar';

const ALL_PRODUCTS = [
  { id: '1', name: 'iPhone 16 Pro Max', category: 'Phones', price: 134999 },
  { id: '2', name: 'MacBook Pro M4', category: 'Laptops', price: 199999 },
  { id: '3', name: 'Sony WH-1000XM5', category: 'Audio', price: 29999 },
  { id: '4', name: 'iPad Air M2', category: 'Tablets', price: 59999 },
  { id: '5', name: 'Apple Watch Ultra 2', category: 'Wearables', price: 89999 },
  { id: '6', name: 'Samsung Galaxy S25 Ultra', category: 'Phones', price: 129999 },
];

const App = () => {
  const [searchQuery, setSearchQuery] = useState('');
  const [selectedCategory, setSelectedCategory] = useState('All');
  const [cartCount, setCartCount] = useState(0);

  // 1. useMemo: Caches the filtered array result
  // Only runs when searchQuery OR selectedCategory changes, NOT when cartCount updates!
  const filteredProducts = useMemo(() => {
    console.log('⚡ Recalculating filtered products...');
    return ALL_PRODUCTS.filter((item) => {
      const matchesSearch = item.name.toLowerCase().includes(searchQuery.toLowerCase());
      const matchesCategory = selectedCategory === 'All' || item.category === selectedCategory;
      return matchesSearch && matchesCategory;
    });
  }, [searchQuery, selectedCategory]);

  // 2. useCallback: Caches the function reference
  // Keeps the exact same function memory address across renders!
  const handleAddToCart = useCallback((productName) => {
    setCartCount((prev) => prev + 1);
    alert(`Added to cart: ${productName}`);
  }, []);

  const categories = ['All', 'Phones', 'Laptops', 'Audio', 'Tablets'];

  return (
    <SafeAreaProvider>
      <StatusBar style="light" backgroundColor="#090d16" />
      <SafeAreaView className="flex-1 bg-slate-950 px-5 pt-3">

        {/* Header with Cart Counter */}
        <View className="flex-row justify-between items-center mb-4">
          <View>
            <Text className="text-2xl font-black text-sky-400">Tech Store ⚡</Text>
            <Text className="text-slate-400 text-xs">useMemo & useCallback Powered</Text>
          </View>
          <View className="bg-slate-900 border border-slate-800 px-3.5 py-2 rounded-xl flex-row items-center gap-2">
            <Text className="text-base">🛒</Text>
            <Text className="text-emerald-400 font-extrabold text-sm">{cartCount}</Text>
          </View>
        </View>

        {/* Search Input */}
        <TextInput
          value={searchQuery}
          onChangeText={setSearchQuery}
          placeholder="Search products..."
          placeholderTextColor="#64748b"
          className="bg-slate-900 border border-slate-800 text-white px-4 py-3 rounded-xl mb-3 text-sm font-medium"
        />

        {/* Category Filter Pills */}
        <View className="flex-row gap-2 mb-4">
          {categories.map((cat) => (
            <Pressable
              key={cat}
              onPress={() => setSelectedCategory(cat)}
```

```
              className={`px-3 py-1.5 rounded-lg border ${
                selectedCategory === cat
                  ? 'bg-sky-500 border-sky-400'
                  : 'bg-slate-900 border-slate-800'
              }`}
            >
              <Text
                className={`text-xs font-bold ${
                  selectedCategory === cat ? 'text-white' : 'text-slate-400'
                }`}
              >
                {cat}
              </Text>
            </Pressable>
          ))}
        </View>

        {/* Filtered Product List */}
        <FlatList
          data={filteredProducts}
          keyExtractor={(item) => item.id}
          ItemSeparatorComponent={() => <View className="h-3" />}
          renderItem={({ item }) => (
            <View className="bg-slate-900 border border-slate-800 p-4 rounded-2xl flex-row justify-between items-center">
              <View className="flex-1 pr-3">
                <Text className="text-white font-bold text-base">{item.name}</Text>
                <Text className="text-sky-400 text-xs mt-0.5 font-semibold">{item.category}</Text>
                <Text className="text-emerald-400 font-extrabold text-sm mt-1">
                  ₹{item.price.toLocaleString('en-IN')}
                </Text>
              </View>

              <Pressable
                onPress={() => handleAddToCart(item.name)}
                className="bg-slate-800 active:bg-sky-500 px-3.5 py-2 rounded-xl border border-slate-700 items-center justify-
              >
                <Text className="text-white font-bold text-xs">+ Add</Text>
              </Pressable>
            </View>
          )}
          showsVerticalScrollIndicator={false}
          contentContainerStyle={{ paddingBottom: 20 }}
        />

      </SafeAreaView>
    </SafeAreaProvider>
  );
};

export default App;
```

Telugu + English Explanation:

• **useMemo**: User cart button (`+ Add`) nokkinappudu `cartCount` state update avthundi. Normal ga ayithe products list malli filter avvali. Kani `useMemo` valla `searchQuery` or `selectedCategory` maarithe thappa, aa heavy filter function malli run avvadu!

• **useCallback**: `handleAddToCart` function memory lo save aypoyi untundi. Parent component re-render aina, prathi item ki kothha function create avvadu. Same function reference share avthundi.

🪝 Level 3.3: The useRef Hook (Imperative Access & Silent Memory)

1. Mental Model: The 2 Superpowers of useRef

Superpower	What it does	Real-World Mobile Example
1. Imperative Component Access	Directly controls native widgets via `.current` methods.	**Auto-focusing text inputs** (like jumping to next box in a 4-digit OTP) or programmatically scrolling a list.
2. Silent Persistent Memory	Stores a value that survives re-renders **without triggering a new render**.	Storing `timerId`, previous state values, or tap tracking flags without freezing the UI.

```
useState:
setCount(5)  ──▶  Triggers Full UI Re-Render 🔄

useRef:
countRef.current = 5  ──▶  Stores value in background SILENTLY (No Re-Render! 🤫)
```

💻 Code Example: 4-Digit Auto-Advancing OTP Input Grid

```
import React, { useState, useRef } from 'react';
import { View, Text, TextInput, Pressable, Keyboard } from 'react-native';
import { SafeAreaProvider, SafeAreaView } from 'react-native-safe-area-context';
import { StatusBar } from 'expo-status-bar';

const App = () => {
  const [otp, setOtp] = useState(['', '', '', '']);

  // 1. Creating 4 imperative references for the 4 input boxes
  const inputRef0 = useRef(null);
  const inputRef1 = useRef(null);
  const inputRef2 = useRef(null);
  const inputRef3 = useRef(null);

  const inputRefs = [inputRef0, inputRef1, inputRef2, inputRef3];

  // 2. Auto-Focus Logic when typing
  const handleChangeText = (text, index) => {
    const newOtp = [...otp];
    newOtp[index] = text;
    setOtp(newOtp);

    // If user entered a digit, automatically focus next input box
    if (text.length === 1 && index < 3) {
      inputRefs[index + 1].current?.focus();
    }

    // If user filled all 4 boxes, dismiss keyboard
    if (index === 3 && text.length === 1) {
      Keyboard.dismiss();
    }
  };

  // 3. Handle Backspace (jumping back to previous input)
  const handleKeyPress = (e, index) => {
    if (e.nativeEvent.key === 'Backspace' && otp[index] === '' && index > 0) {
      inputRefs[index - 1].current?.focus();
    }
  };

  const handleVerify = () => {
    const fullOtp = otp.join('');
    if (fullOtp.length === 4) {
      alert(`Verifying OTP: ${fullOtp} ✅`);
    } else {
      alert('Please enter all 4 digits! ⚠️');
    }
  };

  return (
    <SafeAreaProvider>
      <StatusBar style="light" backgroundColor="#090d16" />
      <SafeAreaView className="flex-1 bg-slate-950 px-6 justify-center">

        {/* Card Container */}
        <View className="bg-slate-900 border border-slate-800 p-6 rounded-3xl items-center shadow-2xl">

          <Text className="text-3xl mb-2">🔢</Text>
          <Text className="text-2xl font-black text-white mb-1">Enter OTP Code</Text>
          <Text className="text-slate-400 text-xs text-center mb-8">
            useRef automatically jumps to the next box as you type.
          </Text>

          {/* 4-Digit Input Boxes Grid */}
          <View className="flex-row gap-3 justify-center mb-8">
            {otp.map((digit, index) => (
              <TextInput
                key={index}
                ref={inputRefs[index]} // Attaching ref handle
                value={digit}
                onChangeText={(text) => handleChangeText(text, index)}
```

```
                onKeyPress={(e) => handleKeyPress(e, index)}
                keyboardType="number-pad"
                maxLength={1}
                selectTextOnFocus
                className={`w-14 h-16 text-center text-2xl font-black rounded-2xl border ${
                  digit
                    ? 'border-sky-400 bg-sky-500/10 text-white'
                    : 'border-slate-800 bg-slate-950 text-slate-400'
                }`}
              />
            ))}
          </View>

          {/* Verify Button */}
          <Pressable
            android_ripple={{ color: '#38bdf840' }}
            onPress={handleVerify}
            className="w-full bg-sky-500 active:bg-sky-600 p-4 rounded-xl items-center"
          >
            <Text className="text-white font-bold text-base">Verify Code 🚀</Text>
          </Pressable>

          {/* Reset & Focus 1st box button */}
          <Pressable
            onPress={() => {
              setOtp(['', '', '', '']);
              inputRefs[0].current?.focus(); // Programmatically focus first box!
            }}
            className="mt-4"
          >
            <Text className="text-slate-500 text-xs font-semibold">Clear & Focus 1st Box</Text>
          </Pressable>

        </View>

      </SafeAreaView>
    </SafeAreaProvider>
  );
};

export default App;
```

🗣️ Telugu + English Explanation:

• **ref={inputRef} & .focus()**: Manam manual ga finger tho prathi box ni touch cheyalsina pani lekunda, `inputRefs[index + 1].current.focus()` vaadi user type cheyagane cursor automatic ga next box loki jump ayyela chesthunnam.

• **Backspace Handling**: `handleKeyPress` lo `e.nativeEvent.key === 'Backspace'` check chesi, current box empty unte cursor ni previous box (`index - 1`) loki jump chesthunnam.

• **Silent Memory**: `useRef` lopalunna value maarina screen malli re-render avvadu, idi native imperative commands (focus, scroll, blur) ki best tool!

🪝 Level 3.4: Custom Mobile Hooks

1. Mental Model: Why Custom Hooks in React Native?

Mobile apps lo chala logic repetitive ga untundi (e.g., keyboard open lo unda leda check cheyadam, search bar lo user type chesthunappudu API calls spam avvakunda **debounce** cheyadam, network connectivity check cheyadam).

> **Golden Rule of Custom Hooks:**
> Custom Hook is just a normal JavaScript function whose name starts with `use` (like `useDebounce`, `useKeyboardVisible`) and can internally use other React hooks (`useState`, `useEffect`, `useRef`).

2. Two Essential Production Custom Hooks

1. `useDebounce(value, delay)`: User search bar lo type chesthunnappudu prathi letter ki API call vellakunda, typing aapi `500ms` aagaka okka saari mathrame search trigger chesthundi.
2. `useKeyboardVisible()`: Phone lo virtual keyboard open lo unda leda live ga detect chesthundi (e.g., keyboard open unte bottom promotional banners or floating buttons ni hide cheyadaniki).

💻 Code Example: Debounced Search & Keyboard Sensor

```
import React, { useState, useEffect } from 'react';
import { View, Text, TextInput, Keyboard, Pressable } from 'react-native';
import { SafeAreaProvider, SafeAreaView } from 'react-native-safe-area-context';
import { StatusBar } from 'expo-status-bar';

// ------------------------------------------------
// 🪝 CUSTOM HOOK 1: useDebounce
// ------------------------------------------------
const useDebounce = (value, delay = 500) => {
  const [debouncedValue, setDebouncedValue] = useState(value);

  useEffect(() => {
    // Set timer to update debounced value after delay
    const timer = setTimeout(() => {
      setDebouncedValue(value);
    }, delay);

    // Clean up timer if user types again before delay finishes
    return () => clearTimeout(timer);
  }, [value, delay]);

  return debouncedValue;
};

// ------------------------------------------------
// 🪝 CUSTOM HOOK 2: useKeyboardVisible
// ------------------------------------------------
const useKeyboardVisible = () => {
  const [isKeyboardVisible, setKeyboardVisible] = useState(false);

  useEffect(() => {
    const showSubscription = Keyboard.addListener('keyboardDidShow', () => {
      setKeyboardVisible(true);
    });
    const hideSubscription = Keyboard.addListener('keyboardDidHide', () => {
      setKeyboardVisible(false);
    });

    // Cleanup native event subscriptions
    return () => {
      showSubscription.remove();
      hideSubscription.remove();
    };
  }, []);

  return isKeyboardVisible;
};

// ------------------------------------------------
// 📱 MAIN COMPONENT
// ------------------------------------------------
const App = () => {
  const [searchTerm, setSearchTerm] = useState('');

  // Using our 2 Custom Hooks!
  const debouncedSearchTerm = useDebounce(searchTerm, 600);
  const isKeyboardOpen = useKeyboardVisible();

  return (
    <SafeAreaProvider>
      <StatusBar style="light" backgroundColor="#090d16" />
      <SafeAreaView className="flex-1 bg-slate-950 px-6 justify-between py-6">

        {/* Top Content */}
        <View>
          <Text className="text-2xl font-black text-sky-400 mb-1">
            Custom Hooks 🪝
          </Text>
          <Text className="text-slate-400 text-xs mb-6">
            Reusable logic: useDebounce & useKeyboardVisible
          </Text>
```

```
        {/* Search Input Box */}
        <View className="bg-slate-900 border border-slate-800 p-4 rounded-2xl mb-6">
          <Text className="text-slate-300 text-xs font-bold uppercase mb-2">
            Live Search Input
          </Text>
          <TextInput
            value={searchTerm}
            onChangeText={setSearchTerm}
            placeholder="Type something fast..."
            placeholderTextColor="#64748b"
            className="bg-slate-950 text-white p-3.5 rounded-xl border border-slate-800 text-sm font-medium"
          />
        </View>

        {/* Comparison Cards */}
        <View className="gap-3">

          {/* Instant Value */}
          <View className="bg-slate-900 border border-slate-800 p-4 rounded-2xl">
            <Text className="text-slate-400 text-xs font-semibold">1. Instant State (updates every keystroke):</Text>
            <Text className="text-amber-400 font-bold text-base mt-1">
              "{searchTerm || '...'}"
            </Text>
          </View>

          {/* Debounced Value */}
          <View className="bg-slate-900 border border-slate-800 p-4 rounded-2xl">
            <Text className="text-slate-400 text-xs font-semibold">2. Debounced Value (waits 600ms):</Text>
            <Text className="text-emerald-400 font-bold text-base mt-1">
              "{debouncedSearchTerm || '...'}"
            </Text>
            <Text className="text-slate-500 text-[11px] mt-1">
              🚀 API call triggers ONLY for this debounced value!
            </Text>
          </View>

          {/* Keyboard Status Sensor */}
          <View className="bg-slate-900 border border-slate-800 p-4 rounded-2xl flex-row justify-between items-center">
            <Text className="text-slate-400 text-xs font-semibold">Software Keyboard:</Text>
            <View className={`px-3 py-1 rounded-full ${isKeyboardOpen ? 'bg-emerald-500/20 border border-emerald-500/30' : '
              <Text className={`text-xs font-bold ${isKeyboardOpen ? 'text-emerald-400' : 'text-slate-400'}`}>
                {isKeyboardOpen ? 'OPEN (Typing)' : 'CLOSED (Hidden)'}
              </Text>
            </View>
          </View>

        </View>
      </View>

      {/* Bottom Banner (Automatically hides when keyboard is open!) */}
      {!isKeyboardOpen ? (
        <View className="bg-sky-500/10 border border-sky-500/30 p-4 rounded-2xl items-center">
          <Text className="text-sky-400 font-bold text-xs">
            💡 Floating Action Bar (Visible only when keyboard is closed)
          </Text>
        </View>
      ) : (
        <Pressable
          onPress={Keyboard.dismiss}
          className="bg-slate-800 p-3 rounded-xl items-center"
        >
          <Text className="text-slate-400 text-xs font-bold">Tap to Close Keyboard ⌨</Text>
        </Pressable>
      )}

    </SafeAreaView>
  </SafeAreaProvider>
 );
};
```

```
export default App;
```

🗣️ Telugu + English Explanation:

• **useDebounce**: User search bar lo `react native` ani fast ga type chesthe, normal ga 12 API calls vellipothayi (prathi letter ki okati). `useDebounce` vaadithe user type cheyadam aape varaku timer reset avthu untundi, typing complete ayina `600ms` tharvatha okke okka API call velthundi. 90% backend server load thagguthundi!

• **useKeyboardVisible**: Mobile keyboard open ayinappudu `Keyboard.addListener` dwara detect chesi `isKeyboardOpen = true` return chesthundi. Bottom lo unde ads, banners or floating bars ni automatically hide chesi screen clean ga unchadaniki idi use avthundi.